Nexierge Mobile - Technical Documentation

Table of Contents

1. Project Overview

Nexierge is a comprehensive hotel operations management mobile application built with Flutter. It provides real-time ticket management, dashboard analytics, user profile management, and activity tracking for hotel staff.

Key Features

• Real-time ticket creation and management

• Dashboard with KPI metrics and "Needs Attention" items

• Multi-department service catalog system

• Push notifications for ticket updates

• Offline-first architecture with local caching

• Multi-language support (English/Spanish)

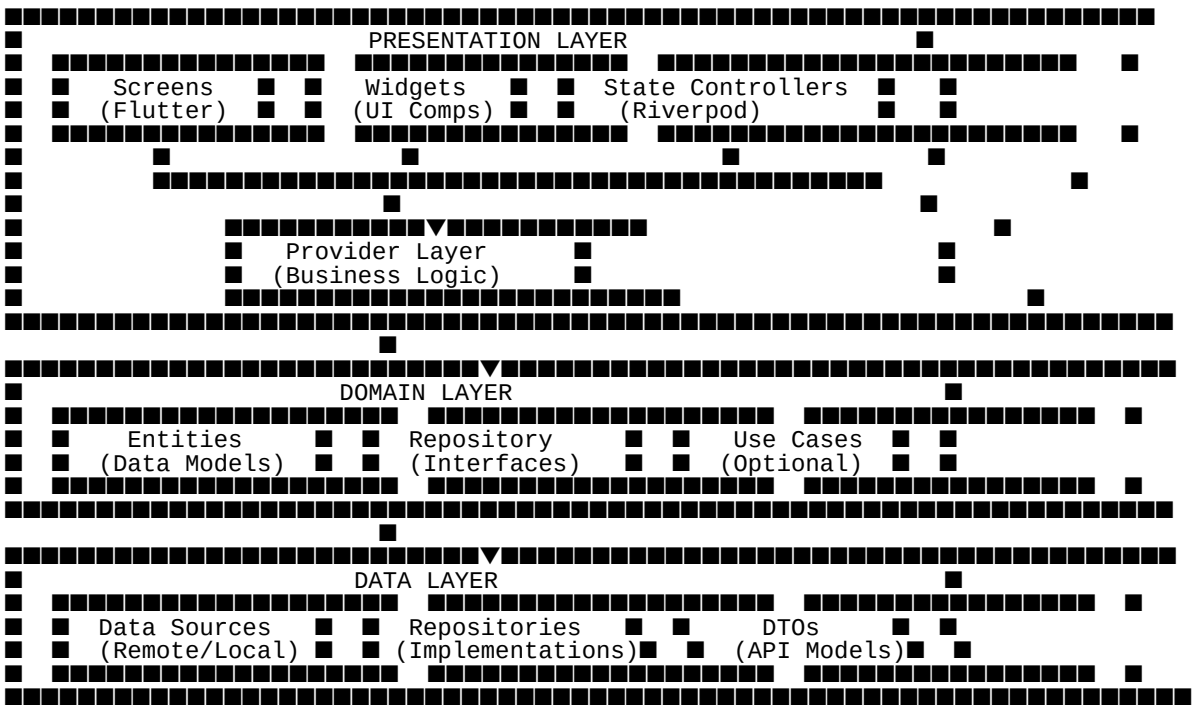
• Role-based access control

Technology Stack

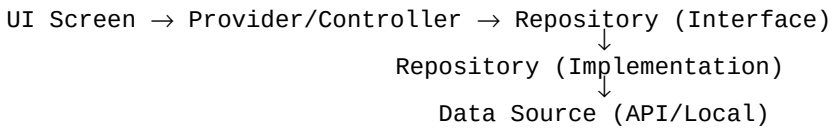
Framework	Flutter 3.9.2+
State Management	Riverpod 2.6.1
Backend API	REST + WebSocket (Socket.IO)
Authentication	JWT with Secure Storage
Push Notifications	Firebase Cloud Messaging (FCM)
Local Database	SharedPreferences + In-Memory Cache
Localization	flutter_localizations + ARB

2. Architecture Overview

2.1 Clean Architecture + MVVM Pattern



2.2 Dependency Flow



Key Principles:

• **Dependency Inversion**: UI depends on abstractions (Repository interfaces)

• **Single Responsibility**: Each class has one reason to change

• **Interface Segregation**: Small, focused interfaces

• **Open/Closed**: Extensible without modifying existing code

3. Project Structure

```
lib/
├── core/                          # Shared infrastructure
│   ├── constants/                 # App constants
│   ├── error/                     # Error handling
│   ├── i18n/                      # Internationalization
│   ├── network/                   # Dio client, interceptors
│   ├── providers/                 # Global providers
│   ├── services/                  # Core services (FCM, Socket, etc.)
│   ├── theme/                     # App theming
│   ├── utils/                     # Utilities
│   └── widgets/                   # Reusable widgets
│
├── features/                      # Feature modules
│   ├── activity/                  # Activity feed
│   ├── auth/                      # Authentication
│   ├── dashboard/                 # Home dashboard
│   ├── fcm/                       # Firebase messaging
│   ├── languages/                 # Language settings
│   ├── modules/                   # Feature modules registry
│   ├── notifications/             # Notification inbox
│   ├── profile/                   # User profile
│   ├── rooms/                     # Room management
│   ├── shell/                     # App shell (bottom nav)
│   └── tickets/                   # Ticket system
│
├── l10n/                          # Localization files (ARB)
    ├── app_en.arb
    └── app_es.arb
```

3.1 Feature Module Structure

Each feature follows the same structure:

```
features/{feature_name}/
├── data/
│   ├── datasources/               # API/Remote data sources
│   ├── models/                    # DTOs, data models
│   └── repositories/            # Repository implementations
├── domain/
│   └── entities/                  # Domain entities
```

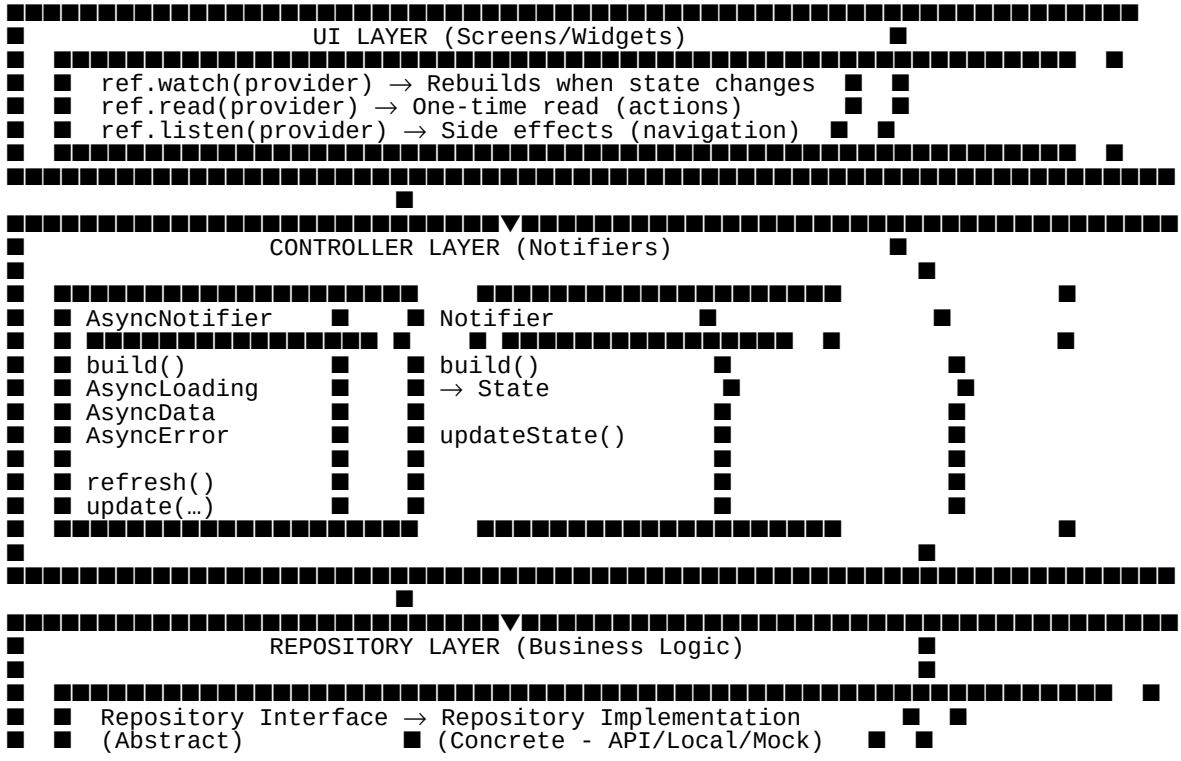
```
■   ■■■ repositories/              # Repository interfaces (abstract)
■■■ presentation/
    ■■■ providers/                 # State controllers (Riverpod)
    ■■■ screens/                   # UI screens
    ■■■ widgets/                   # Feature-specific widgets
```

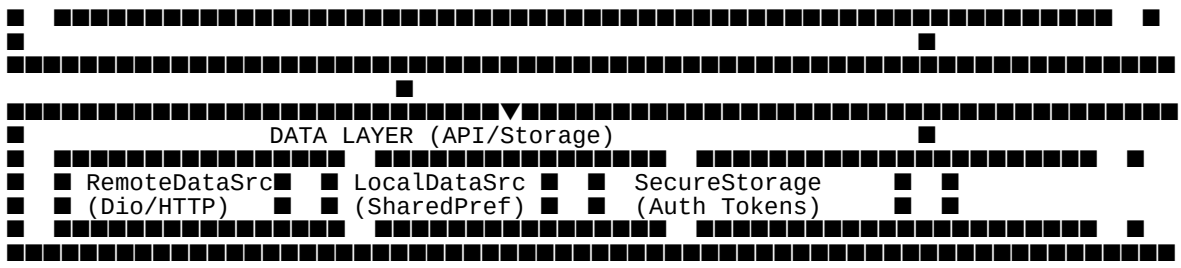
4. State Management (Riverpod)

4.1 Provider Types Used

Dependency Injection	`Provider`	`dioClientProvider`, `authRepositoryProvider`
Simple UI State	`StateProvider`	`ticketBusyProvider`
Async API Data	`AsyncNotifier`	`dashboardCountsControllerProvider`
Sync Business Logic	`Notifier`	`notificationInboxControllerProvider`
Auto-dispose State	`AutoDisposeAsyncNotifier`	`userProfileControllerProvider`
Paginated Lists	`FamilyAsyncNotifier`	`ticketsPagedNotifierProvider`

4.2 State Management Architecture





4.3 Example: Dashboard Counts Controller

```
// Controller definition
class DashboardCountsController extends AsyncNotifier<DashboardCounts> {
  late DashboardRepository _repo;

  @override
  Future<DashboardCounts> build() async {
    // Watches auth session - rebuilds when session changes
    final session = await ref.watch(authSessionControllerProvider.future);
    _repo = ref.read(dashboardRepositoryProvider);
    return _repo.fetchDashboardCounts(hotelUserId: session?.hotelUserId);
  }

  Future<void> refresh() async {
    state = const AsyncLoading();
    state = await AsyncValue.guard(() async {
      final session = await ref.read(authSessionControllerProvider.future);
      return _repo.fetchDashboardCounts(hotelUserId: session?.hotelUserId);
    });
  }
}

// Provider registration
final dashboardCountsControllerProvider = AsyncNotifierProvider<
  DashboardCountsController,
  DashboardCounts
>(() => DashboardCountsController());
```

5. Data Layer Architecture

5.1 Repository Pattern

```
// Domain Layer - Interface (Abstract)
abstract class TicketsRepository {
  Stream<List<Ticket>> watchAll();
  Future<TicketDetail> fetchTicketDetails({required String ticketId});
  Future<void> updateStatus({required String ticketId, required TicketStatus newStatus});
}

// Data Layer - Implementation
class _TicketRepositoryImpl implements TicketsRepository {
  final TicketRemoteDataSource _remote;
  final TicketLocalDataSource _local;

  _TicketRepositoryImpl(this._remote, this._local);

  @override
  Future<TicketDetail> fetchTicketDetails({required String ticketId}) async {
```

```
    // Try cache first
    final cached = await _local.getTicket(ticketId);
    if (cached != null) return cached;

    // Fetch from API
    final dto = await _remote.fetchTicketDetails(ticketId: ticketId);
    final entity = dto.toEntity();

    // Update cache
    await _local.saveTicket(entity);
    return entity;
  }
}
```

5.2 Data Flow

```
User Action
    ↓
Controller/Provider
    ↓
Repository (Business Logic)
    ↓
Data Source (API Call)
    ↓
DTO (JSON → Dart)
    ↓
Entity (Domain Model)
    ↓
UI Update (via State)
```

5.3 API Client (Dio)

```
// lib/core/network/api_client.dart
class DioClient {
  late final Dio _dio;

  DioClient() {
    _dio = Dio(BaseOptions(
      baseUrl: AppConstants.apiBaseUrl,
      connectTimeout: const Duration(seconds: 30),
      receiveTimeout: const Duration(seconds: 30),
    ));

    _dio.interceptors.addAll([
      AuthInterceptor(),      // Add JWT token
      LoggingInterceptor(),   // Log requests/responses
      ErrorInterceptor(),     // Handle API errors
    ]);
  }
}
```

6. Feature Modules

6.1 Dashboard Feature

Responsibility: Home screen with KPI metrics and "Needs Attention" items

```
dashboard/
■■■ data/
■   ■■■ datasources/
■   ■   ■■■ dashboard_remote_data_source.dart
■   ■■■ models/
■   ■   ■■■ dashboard_counts_dto.dart
■   ■   ■■■ needs_attention_item_dto.dart
■   ■■■ repositories/
■       ■■■ dashboard_repository.dart
■■■ domain/
■   ■■■ entities/
■   ■   ■■■ dashboard_counts.dart
■   ■   ■■■ needs_attention_item.dart
■   ■■■ repositories/
■       ■■■ dashboard_repository.dart (abstract)
■■■ presentation/
    ■■■ providers/
    ■   ■■■ dashboard_bootstrap_controller.dart
    ■   ■■■ dashboard_counts_controller.dart
    ■   ■■■ needs_attention_controller.dart
    ■■■ screens/
    ■   ■■■ dashboard_screen.dart
    ■■■ widgets/
        ■■■ dashboard_greeting.dart
        ■■■ dashboard_stats_grid.dart
        ■■■ dashboard_stats_compact.dart
        ■■■ needs_attention_list.dart
```

Key Components:

• `DashboardBootstrapController`: Initializes dashboard data on app start

• `DashboardCountsController`: Manages KPI metrics (incoming, accepted, in-progress, overdue)

• `NeedsAttentionController`: Manages priority items requiring staff attention

6.2 Tickets Feature

Responsibility: Ticket management system (CRUD, status updates, filtering)

```
tickets/
■■■ data/
■   ■■■ datasources/
■   ■   ■■■ ticket_remote_data_source.dart
■   ■   ■■■ guest_stay_remote_data_source.dart
■   ■■■ models/
■   ■   ■■■ ticket_dto.dart
■   ■   ■■■ ticket_detail_dto.dart
■   ■   ■■■ service_catalog_dto.dart
■   ■■■ repositories/
■   ■   ■■■ ticket_repository.dart
■   ■   ■■■ mock_tickets_repository.dart
■   ■   ■■■ guest_stay_repository.dart
■   ■■■ seeds/
■       ■■■ mock_seed.dart
■■■ domain/
■   ■■■ entities/
■   ■   ■■■ ticket.dart
■   ■   ■■■ ticket_detail.dart
■   ■   ■■■ service_catalog.dart
■   ■■■ repositories/
■       ■■■ tickets_repository.dart (abstract)
■■■ presentation/
    ■■■ providers/
    ■   ■■■ tickets_paged_notifier.dart
    ■   ■■■ ticket_detail_api_controller.dart
    ■   ■■■ universal_create_controller.dart
    ■   ■■■ catalog_create_controller.dart
```

```
│   ├── manual_create_controller.dart
│   └── service_catalogs_provider.dart
├── screens/
│   ├── tickets_list_screen.dart
│   ├── ticket_detail_screen.dart
│   ├── universal_create_screen.dart
│   ├── catalog_create_screen.dart
│   └── manual_create_screen.dart
└── widgets/
    ├── ticket_card.dart
    ├── ticket_status_badge.dart
    ├── ticket_filter_chips.dart
    └── ...
```

Key Components:

• `TicketsPagedNotifier`: Paginated ticket list with infinite scroll

• `UniversalCreateController`: Creates tickets from search

• `CatalogCreateController`: Creates tickets from service catalog

• `ManualCreateController`: Creates custom tickets

6.3 Auth Feature

Responsibility: Authentication, session management, user profile

```
auth/
├── data/
│   ├── datasources/
│   │   └── auth_remote_data_source.dart
│   ├── models/
│   │   ├── login_request_dto.dart
│   │   └── auth_session_dto.dart
│   └── repositories/
│       ├── auth_repository_impl.dart
│       └── auth_session_storage.dart
├── domain/
│   ├── entities/
│   │   ├── auth_session.dart
│   │   └── login_credentials.dart
│   └── repositories/
│       └── auth_repository.dart (abstract)
└── presentation/
    ├── providers/
    │   ├── auth_session_controller.dart
    │   └── login_controller.dart
    └── screens/
        └── login_screen.dart
```

Key Components:

• `AuthSessionController`: Manages JWT token, session persistence

• `AuthSessionStorage`: Secure storage for tokens (flutter_secure_storage)

• `LoginController`: Handles login form submission

6.4 Notifications Feature

Responsibility: Push notifications, FCM handling, notification inbox

```
notifications/
├── data/
│   ├── datasources/
│   │   └── notification_remote_datasource.dart
│   └── repositories/
│       └── notification_repository_impl.dart
├── domain/
│   ├── entities/
│   │   └── notification_entity.dart
│   └── repositories/
│       └── i_notification_repository.dart
└── presentation/
    ├── providers/
    │   ├── notification_inbox_controller.dart
    │   └── notification_notifier.dart
    └── screens/
        └── notifications_inbox_screen.dart
```

Key Components:

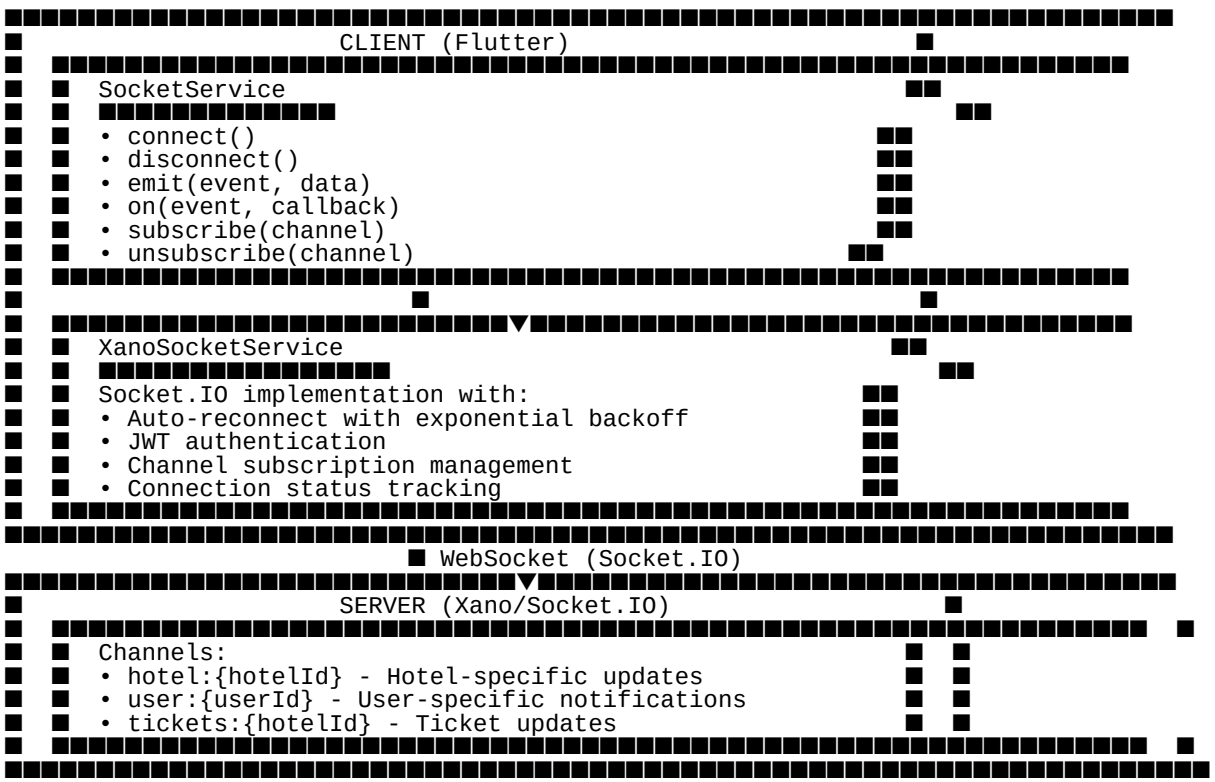
• `NotificationService` (core/services): FCM initialization, background handlers

• `NotificationInboxController`: Manages in-app notification list

• `_onBackgroundMessage`: Top-level function for background FCM messages

7. Real-time Communication

7.1 WebSocket Architecture (Socket.IO)



7.2 Real-time Ticket Updates

```
// lib/features/tickets/data/datasources/ticket_remote_data_source.dart
class TicketRemoteDataSource {
  final SocketService _socket;

  TicketRemoteDataSource(this._socket);

  void listenToTicketUpdates(String hotelId, Function(TicketDto) onUpdate) {
    _socket.subscribe('tickets:$hotelId');
    _socket.on('ticket:updated', (data) {
      final dto = TicketDto.fromJson(data);
      onUpdate(dto);
    });
  }

  void dispose() {
    _socket.unsubscribe('tickets:$hotelId');
  }
}
```

7.3 Connection Lifecycle

```
App Start
    ↓
Check Auth Session
    ↓
Connect Socket (with JWT)
    ↓
Subscribe to Hotel Channel
    ↓
Listen for Real-time Updates
    ↓
App Background → Disconnect (optional)
    ↓
App Foreground → Reconnect + Resubscribe
```

8. Authentication & Security

8.1 JWT Token Flow

```
Login Screen
    ↓
Submit Credentials
    ↓
POST /auth/login
    ↓
Receive JWT Token + User Profile
    ↓
Store Token (Secure Storage)
    ↓
Store Session (SharedPreferences)
    ↓
Initialize Authenticated Services
    ↓
Navigate to Dashboard
```

8.2 Token Refresh

```
// lib/core/network/interceptors/auth_interceptor.dart
class AuthInterceptor extends Interceptor {
  @override
  void onRequest(RequestOptions options, RequestInterceptorHandler handler) {
    final token = _secureStorage.read('auth_token');
    if (token != null) {
      options.headers['Authorization'] = 'Bearer $token';
    }
    handler.next(options);
  }

  @override
  void onError(DioException err, ErrorInterceptorHandler handler) {
    if (err.response?.statusCode == 401) {
      // Token expired - trigger refresh or logout
      _handleTokenExpiration();
    }
    handler.next(err);
  }
}
```

8.3 Secure Storage

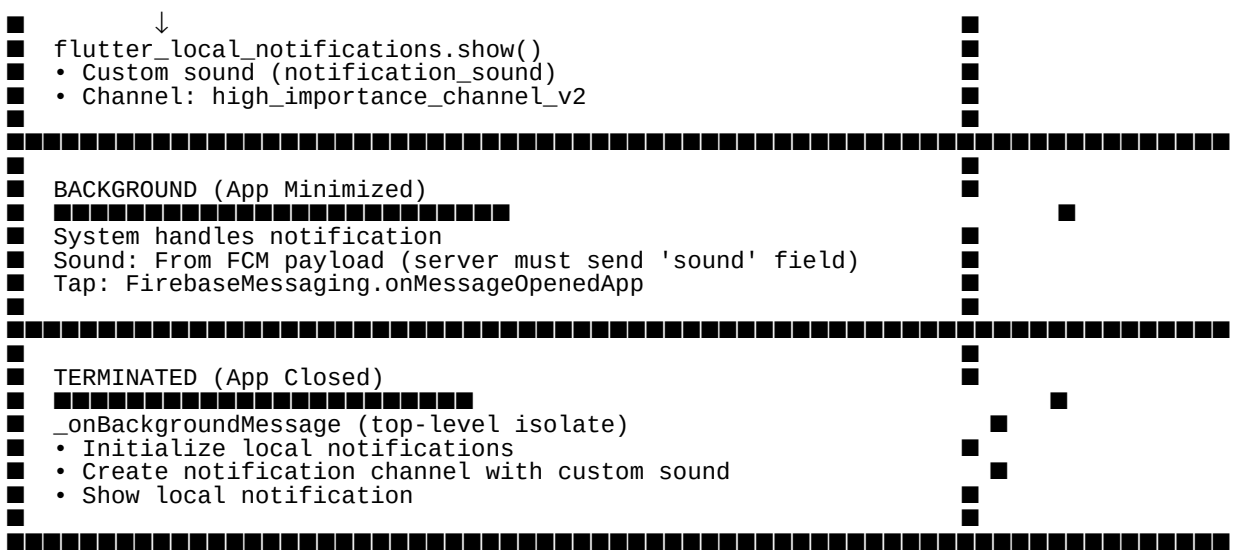
```
// Tokens use flutter_secure_storage (iOS Keychain / Android Keystore)
class AuthSessionStorage {
  final _secureStorage = const FlutterSecureStorage();
  final _prefs = SharedPreferences.getInstance();

  Future<void> saveSession(AuthSession session) async {
    // Token in secure storage
    await _secureStorage.write(key: 'auth_token', value: session.token);
    // Session data in SharedPreferences
    await _prefs.setString('session_data', jsonEncode(session.toJson()));
  }

  Future<AuthSession?> getSession() async {
    final token = await _secureStorage.read(key: 'auth_token');
    final data = _prefs.getString('session_data');
    if (token == null || data == null) return null;
    return AuthSession.fromJson(jsonDecode(data))..token = token;
  }
}
```

9. Push Notifications

9.1 FCM Architecture



9.2 Required Server Payload

For custom sound to work in **background/terminated** states:

Android:

```
{
  "notification": {
    "title": "New Ticket",
    "body": "Ticket #12345 needs attention",
    "sound": "notification_sound"
  },
  "data": {
    "ticketId": "12345",
    "type": "ticket_created"
  }
}
```

iOS:

```
{
  "notification": {
    "title": "New Ticket",
    "body": "Ticket #12345 needs attention",
    "sound": "notification_sound.caf"
  },
  "data": {
    "ticketId": "12345"
  }
}
```

9.3 Sound Files Location

Android	**`android/app/src/main/res/raw/notification_sound.mp3`**	**MP3**
iOS	`ios/Runner/notification_sound.caf`	CAF (Core Audio Format)

10. Internationalization

10.1 ARB (Application Resource Bundle) Structure

```
l10n/
├── app_en.arb          # English (default)
└── app_es.arb          # Spanish
```

10.2 Usage in Code

```
// Extension method for easy access
extension L10nExtension on BuildContext {
  AppLocalizations get l10n => AppLocalizations.of(this)!;
}

// Usage in widgets
Text(context.l10n.dashboardGreeting)
Text(context.l10n.ticketStatusInProgress)
```

10.3 Adding New Strings

1. Add to `app_en.arb`:

```
{
  "@@locale": "en",
  "newFeatureTitle": "New Feature",
  "@newFeatureTitle": {
    "description": "Title for the new feature screen"
  }
}
```

2. Add to `app_es.arb`:

```
{
  "@@locale": "es",
  "newFeatureTitle": "Nueva Característica"
}
```

3. Run code generation:

```
flutter gen-l10n
```

11. Testing Strategy

11.1 Test Structure

```
test/
■■■ core/
■   ■■■ network/
■   ■■■ services/
■   ■■■ utils/
■■■ features/
■   ■■■ auth/
■   ■■■ dashboard/
■   ■■■ tickets/
■   ■■■ profile/
■■■ fixtures/
■   ■■■ (JSON test data)
■■■ mocks/
    ■■■ (Mock classes)
```

11.2 Unit Testing Pattern

```
// Repository test
group('TicketRepository', () {
  late TicketRepository repository;
  late MockTicketRemoteDataSource mockRemote;
  late MockTicketLocalDataSource mockLocal;

  setUp(() {
    mockRemote = MockTicketRemoteDataSource();
    mockLocal = MockTicketLocalDataSource();
    repository = TicketRepositoryImpl(mockRemote, mockLocal);
  });

  test('fetchTicketDetails returns cached data when available', () async {
    // Arrange
    final cachedTicket = Ticket(id: '1', title: 'Cached');
    when(mockLocal.getTicket('1')).thenAnswer((_) async => cachedTicket);

    // Act
    final result = await repository.fetchTicketDetails(ticketId: '1');

    // Assert
    expect(result, equals(cachedTicket));
    verifyNever(mockRemote.fetchTicketDetails(ticketId: '1'));
  });
});
```

11.3 Widget Testing

```
// Screen test
testWidgets('LoginScreen shows error on invalid credentials', (tester) async {
  await tester.pumpWidget(
    ProviderScope(
      overrides: [
        loginControllerProvider.overrideWith((ref) => MockLoginController()),
      ],
      child: const MaterialApp(home: LoginScreen()),
    ),
  );

  await tester.enterText(find.byType(TextField).first, 'invalid');
  await tester.tap(find.byType(ElevatedButton));
  await tester.pumpAndSettle();

  expect(find.text('Invalid credentials'), findsOneWidget);
});
```

12. Development Guidelines

12.1 Code Organization Rules

1. **Feature-First Structure**: All code for a feature lives in its folder
2. **No Cross-Feature Imports**: Features communicate via providers only
3. **Interface Segregation**: Keep interfaces small and focused
4. **State Immutability**: Never mutate state directly, use `copyWith`

12.2 Naming Conventions

Controllers	**`{Feature}Controller`**	**`DashboardCountsController`**
Repositories	`{Feature}Repository`	`TicketsRepository`
Data Sources	`{Feature}RemoteDataSource`	`TicketRemoteDataSource`
Entities	PascalCase, no suffix	`Ticket`, `UserProfile`
DTOs	`{Name}Dto`	`TicketDto`, `LoginRequestDto`
Providers	`{name}Provider`	`authSessionControllerProvider`
Screens	`{Feature}Screen`	`DashboardScreen`
Widgets	Descriptive PascalCase	`TicketStatusBadge`

12.3 Provider Declaration Pattern

```
// 1. Declare provider alongside controller
final myControllerProvider = AsyncNotifierProvider<MyController, MyState>(
  () => MyController(),
);

// 2. Controller gets dependencies via ref.read in build()
class MyController extends AsyncNotifier<MyState> {
  @override
  Future<MyState> build() async {
    final repo = ref.read(myRepositoryProvider);
    final session = await ref.watch(authSessionControllerProvider.future);
    return repo.fetchData(session?.hotelId);
  }
}
```

12.4 Error Handling

```
try {
```

```
  final result = await _repository.fetchData();
  return result;
} on DioException catch (e) {
  if (e.response?.statusCode == 401) {
    throw AuthException('Session expired');
  } else if (e.response?.statusCode == 500) {
    throw ServerException('Server error');
  }
  throw NetworkException('Network error: ${e.message}');
} catch (e) {
  throw AppException('Unexpected error: $e');
}
```

Appendix A: Dependencies

```
dependencies:
  # Flutter SDK
  flutter:
    sdk: flutter
  flutter_localizations:
    sdk: flutter

  # State Management
  flutter_riverpod: ^2.6.1
  riverpod_annotation: ^2.6.1

  # Firebase
  firebase_core: ^3.6.0
  firebase_messaging: ^15.1.3

  # Networking
  dio: ^5.7.0
  socket_io_client: ^2.0.3+1
  web_socket_channel: ^2.4.0

  # Local Storage
  shared_preferences: ^2.3.2
  flutter_secure_storage: ^9.2.2

  # Notifications
  flutter_local_notifications: ^17.2.2

  # UI
  google_fonts: ^6.2.1
  shimmer: ^3.0.0
  lucide_icons_flutter: ^3.0.0

  # Media
  image_picker: ^1.1.2
  flutter_image_compress: ^2.3.0
  audioplayers: ^6.1.0

  # Permissions
  permission_handler: ^11.3.1

  # Utilities
  intl: ^0.20.2
  go_router: ^14.2.7
  json_annotation: ^4.9.0

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^5.0.0
  build_runner: ^2.4.13
  riverpod_generator: ^2.6.4
  riverpod_lint: ^2.6.3
  custom_lint: ^0.7.0
  json_serializable: ^6.9.0
```

Appendix B: Environment Setup

Prerequisites

• Flutter SDK 3.9.2 or higher

• Dart 3.0 or higher

• Android Studio / Xcode

• Firebase project configured

Setup Steps

1. **Clone Repository**

```
git clone <repository-url>
cd nexierge_mobile
```

2. **Install Dependencies**

```
flutter pub get
```

3. **Configure Firebase**

• Add `google-services.json` (Android) to `android/app/`

• Add `GoogleService-Info.plist` (iOS) to `ios/Runner/`

4. **Generate Code**

```
flutter pub run build_runner build --delete-conflicting-outputs
flutter gen-l10n
```

5. **Run App**

```
flutter run
```

Document Version: 1.0

Last Updated: May 2026